

```
import os
from typing import List, Tuple, Dict

import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence, pad_packed_sequence

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cpu

▼ Task 1.1

```
from google.colab import files
import tarfile
import gzip
import shutil
import glob

# 1. Upload file .gz từ máy bạn lên Colab
uploaded = files.upload()

data_root = "data"
os.makedirs(data_root, exist_ok=True)

for fname in uploaded.keys():
    path = fname
    print("Đã upload:", path)

    # Trường hợp là .tar.gz hoặc .tgz (chứa nhiều file bên trong)
    if tarfile.is_tarfile(path):
        print("Giải nén tar file vào thư mục", data_root)
        with tarfile.open(path, "r:*") as tar:
            tar.extractall(data_root)

    # Trường hợp file .conllu.gz (một file nén đơn)
    elif fname.endswith(".gz"):
        out_name = os.path.join(
            data_root,
            os.path.splitext(fname)[0] # bỏ đuôi .gz
        )
        print(f"Giải nén gzip {fname} -> {out_name}")
        with gzip.open(path, "rb") as f_in, open(out_name, "wb") as f_out:
            shutil.copyfileobj(f_in, f_out)

    # Các file khác thì copy vào data_root (phòng hờ)
    else:
        shutil.copy(path, data_root)

# 2. Tìm file train/dev .conllu trong thư mục data_root
train_candidates = glob.glob(os.path.join(data_root, "**", "*-ud-train.conllu"), recursive=True)
dev_candidates = glob.glob(os.path.join(data_root, "**", "*-ud-dev.conllu"), recursive=True)

train_file = train_candidates[0] if train_candidates else None
dev_file = dev_candidates[0] if dev_candidates else None

print("train_file:", train_file)
print("dev_file :", dev_file)

if train_file is None or dev_file is None:
    raise FileNotFoundError(
        "Không tìm được en_ewt-ud-train.conllu hoặc en_ewt-ud-dev.conllu sau khi giải nén.\n"
        "Hãy kiểm tra lại file UD bạn upload và cấu trúc thư mục trong đó."
    )
```

Chọn tệp UD_English-EWT (1).tar.gz

UD_English-EWT (1).tar.gz(application/x-gzip) - 3729474 bytes, last modified: 19/11/2025 - 100% done

Saving UD_English-EWT (1).tar.gz to UD_English-EWT (1).tar (2).gz

Đã upload: UD_English-EWT (1).tar (2).gz

Giải nén tar file vào thư mục data

train_file: data/UD_English-EWT/en_ewt-ud-train.conllu

dev_file : data/UD_English-EWT/en_ewt-ud-dev.conllu

/tmp/ipython-input-319435514.py:21: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject f
tar.extractall(data_root)

Task 1.1

```
def load_conllu(file_path: str) -> List[List[Tuple[str, str]]]:
```

```
    """
```

```
    Đọc file .conllu và trả về list các câu.
```

```
    Mỗi câu là list các cặp (word, upos_tag).
```

```
    """
```

```
    sentences = []
```

```
    current_sentence = []
```

```
    with open(file_path, "r", encoding="utf-8") as f:
```

```
        for line in f:
```

```
            line = line.strip()
```

```
            # Dòng trống: kết thúc một câu
```

```
            if not line:
```

```
                if current_sentence:
```

```
                    sentences.append(current_sentence)
```

```
                    current_sentence = []
```

```
                continue
```

```
            # Dòng comment trong CoNLL-U
```

```
            if line.startswith("#"):
```

```
                continue
```

```
            cols = line.split("\t")
```

```
            if len(cols) < 4:
```

```
                continue
```

```
            token_id = cols[0]
```

```
            # Bỏ multiword token dạng "1-2" hoặc "3.1"
```

```
            if "-" in token_id or "." in token_id:
```

```
                continue
```

```
            word = cols[1]      # FORM
```

```
            upos_tag = cols[3] # UPOS
```

```
            current_sentence.append((word, upos_tag))
```

```
    if current_sentence:
```

```
        sentences.append(current_sentence)
```

```
    return sentences
```

```
# Dùng các đường dẫn đã tìm ở Cell 2
```

```
print("Đang đọc train từ:", train_file)
```

```
print("Đang đọc dev   từ:", dev_file)
```

```
train_sentences = load_conllu(train_file)
```

```
dev_sentences   = load_conllu(dev_file)
```

```
print("Số câu train:", len(train_sentences))
```

```
print("Số câu dev  :", len(dev_sentences))
```

```
print("Ví dụ 1 câu train:", train_sentences[0][:10])
```

```
t-ud-train.conllu
```

```
t-ud-dev.conllu
```

```

PUNCT'), ('Zaman', 'PROPN'), (':', 'PUNCT'), ('American', 'ADJ'), ('forces', 'NOUN'), ('killed', 'VERB'), ('Shaikh', 'PROPN'), ('

```

Task 1.2 - Tạo vocab

```

def build_vocab(
    sentences: List[List[Tuple[str, str]]]
) -> Tuple[Dict[str, int], Dict[str, int]]:
    """
    Tạo word_to_ix và tag_to_ix từ tập train.
    - word_to_ix có token đặc biệt <UNK>.
    - tag_to_ix có token đặc biệt <PAD> để dùng cho padding nhãn.
    """
    word_to_ix = {}
    tag_to_ix = {"<PAD>": 0} # padding tag

    word_to_ix["<UNK>"] = 0
    next_word_idx = 1
    next_tag_idx = 1 # 0 đã dành cho <PAD>

    for sent in sentences:
        for word, tag in sent:
            if word not in word_to_ix:
                word_to_ix[word] = next_word_idx
                next_word_idx += 1
            if tag not in tag_to_ix:
                tag_to_ix[tag] = next_tag_idx
                next_tag_idx += 1

    return word_to_ix, tag_to_ix

word_to_ix, tag_to_ix = build_vocab(train_sentences)

print("Kích thước word_to_ix:", len(word_to_ix))
print("Kích thước tag_to_ix :", len(tag_to_ix))

```

```

Kích thước word_to_ix: 19674
Kích thước tag_to_ix : 18

```

Task 2 — Chuẩn bị DataLoader với padding

```

class POSDataset(Dataset):
    """
    Dataset cho POS Tagging.
    Mỗi phần tử là (sentence_indices, tag_indices)
    """

    def __init__(self, sentences, word_to_ix, tag_to_ix):
        self.sentences = sentences
        self.word_to_ix = word_to_ix
        self.tag_to_ix = tag_to_ix
        self.unk_idx = word_to_ix["<UNK>"]
        self.pad_tag_idx = tag_to_ix["<PAD>"]

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        sent = self.sentences[idx]

        words = []
        tags = []
        for w, t in sent:
            words.append(self.word_to_ix.get(w, self.unk_idx))
            tags.append(self.tag_to_ix[t])

        words_tensor = torch.tensor(words, dtype=torch.long)
        tags_tensor = torch.tensor(tags, dtype=torch.long)

        return words_tensor, tags_tensor

```

```

def collate_fn(batch):
    """
    batch: list[(words_tensor, tags_tensor)]
    Trả về:
        padded_words: (batch, max_len)
        padded_tags : (batch, max_len)
        lengths      : (batch,) - độ dài thật.
    """
    words_list, tags_list = zip(*batch)

    lengths = torch.tensor([len(s) for s in words_list], dtype=torch.long)

    unk_idx = word_to_ix["<UNK>"]
    pad_tag_idx = tag_to_ix["<PAD>"]

    padded_words = pad_sequence(
        words_list, batch_first=True, padding_value=unk_idx
    )
    padded_tags = pad_sequence(
        tags_list, batch_first=True, padding_value=pad_tag_idx
    )

    return padded_words, padded_tags, lengths

train_dataset = POSDataset(train_sentences, word_to_ix, tag_to_ix)
dev_dataset   = POSDataset(dev_sentences, word_to_ix, tag_to_ix)

BATCH_SIZE = 32

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_fn
)
dev_loader = DataLoader(
    dev_dataset, batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_fn
)

print("Số batch train:", len(train_loader))
print("Số batch dev  :", len(dev_loader))

```

```

Số batch train: 392
Số batch dev  : 63

```

Task 3 — Xây dựng mô hình RNN POS Tagging

```

class SimpleRNNForTokenClassification(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        tagset_size: int,
        embedding_dim: int = 100,
        hidden_dim: int = 128,
        num_layers: int = 1,
        bidirectional: bool = False,
    ):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.bidirectional = bidirectional
        self.num_directions = 2 if bidirectional else 1

        self.rnn = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=bidirectional,
        )

        self.fc = nn.Linear(hidden_dim * self.num_directions, tagset_size)

```

```
def forward(self, x, lengths):
    """
    x: (batch, seq_len)
    lengths: (batch,) - độ dài thật của từng câu (chưa pad)
    """
    embeds = self.embedding(x) # (batch, seq_len, embedding_dim)

    packed = pack_padded_sequence(
        embeds, lengths.cpu(), batch_first=True, enforce_sorted=False
    )
    packed_output, _ = self.rnn(packed)
    rnn_output, _ = pad_packed_sequence(
        packed_output, batch_first=True
    ) # (batch, seq_len, hidden_dim * num_directions)

    logits = self.fc(rnn_output) # (batch, seq_len, tagset_size)
    return logits
```

✓ Task 4.1 — Khai báo mô hình, loss, optimizer

```
vocab_size = len(word_to_ix)
tagset_size = len(tag_to_ix)

model = SimpleRNNForTokenClassification(
    vocab_size=vocab_size,
    tagset_size=tagset_size,
    embedding_dim=100,
    hidden_dim=128,
    num_layers=1,
    bidirectional=False,
).to(device)

pad_tag_idx = tag_to_ix["<PAD>"]

criterion = nn.CrossEntropyLoss(ignore_index=pad_tag_idx)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

print(model)
```

```
SimpleRNNForTokenClassification(
  (embedding): Embedding(19674, 100)
  (rnn): RNN(100, 128, batch_first=True)
  (fc): Linear(in_features=128, out_features=18, bias=True)
)
```

✓ Task 5.1 — Tính accuracy

```
def evaluate(model, data_loader):
    model.eval()
    total_tokens = 0
    correct_tokens = 0

    with torch.no_grad():
        for sentences, tags, lengths in data_loader:
            sentences = sentences.to(device)
            tags = tags.to(device)
            lengths = lengths.to(device)

            logits = model(sentences, lengths) # (batch, seq_len, tagset_size)
            preds = torch.argmax(logits, dim=-1) # (batch, seq_len)

            pad_tag_idx = tag_to_ix["<PAD>"]
            mask = tags != pad_tag_idx # (batch, seq_len)

            correct_tokens += ((preds == tags) & mask).sum().item()
            total_tokens += mask.sum().item()

    acc = correct_tokens / total_tokens if total_tokens > 0 else 0.0
    return acc
```

Task 4.2 — Huấn luyện mô hình

```

NUM_EPOCHS = 5

best_dev_acc = 0.0
best_state_dict = None

for epoch in range(1, NUM_EPOCHS + 1):
    model.train()
    total_loss = 0.0

    for sentences, tags, lengths in train_loader:
        sentences = sentences.to(device)
        tags = tags.to(device)
        lengths = lengths.to(device)

        optimizer.zero_grad()

        logits = model(sentences, lengths) # (batch, seq_len, tagset_size)

        batch_size, seq_len, num_tags = logits.size()
        logits_flat = logits.view(batch_size * seq_len, num_tags)
        tags_flat = tags.view(batch_size * seq_len)

        loss = criterion(logits_flat, tags_flat)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    train_acc = evaluate(model, train_loader)
    dev_acc = evaluate(model, dev_loader)

    print(
        f"Epoch {epoch:02d} | Loss: {avg_loss:.4f} | "
        f"Train Acc: {train_acc:.4f} | Dev Acc: {dev_acc:.4f}"
    )

    if dev_acc > best_dev_acc:
        best_dev_acc = dev_acc
        best_state_dict = model.state_dict()

print("Best Dev Accuracy:", best_dev_acc)

if best_state_dict is not None:
    model.load_state_dict(best_state_dict)

```

```

Epoch 01 | Loss: 1.1288 | Train Acc: 0.7710 | Dev Acc: 0.7544
Epoch 02 | Loss: 0.6273 | Train Acc: 0.8352 | Dev Acc: 0.8083
Epoch 03 | Loss: 0.4714 | Train Acc: 0.8759 | Dev Acc: 0.8372
Epoch 04 | Loss: 0.3722 | Train Acc: 0.9009 | Dev Acc: 0.8544
Epoch 05 | Loss: 0.3046 | Train Acc: 0.9205 | Dev Acc: 0.8683
Best Dev Accuracy: 0.8683446657918804

```

```

ix_to_tag = {v: k for k, v in tag_to_ix.items()}

def predict_sentence(sentence: str, model, word_to_ix, ix_to_tag):
    """
    sentence: câu dạng string, ví dụ: "I love NLP"
    """
    model.eval()

    tokens = sentence.strip().split()
    unk_idx = word_to_ix["<UNK>"]

    word_indices = [word_to_ix.get(w, unk_idx) for w in tokens]
    words_tensor = torch.tensor(word_indices, dtype=torch.long).unsqueeze(0)
    lengths = torch.tensor([len(tokens)], dtype=torch.long)

    words_tensor = words_tensor.to(device)
    lengths = lengths.to(device)

```

```
with torch.no_grad():
    logits = model(words_tensor, lengths)
    preds = torch.argmax(logits, dim=-1).squeeze(0)

pred_tags = [ix_to_tag[int(idx)] for idx in preds]

for w, t in zip(tokens, pred_tags):
    print(f"{w}\t{t}")
```